Training at the Championship Level

Virtual contests, upsolving, team roles, and the notebook

Learning goals

- Know what changes between a regional and a national championship
- Run a weekly loop: learn → implement → contest → upsolve → review
- Run a useful virtual contest and post-mortem
- Organize three people around one computer
- Build and maintain a *tested* team notebook
- Have a stress-testing kit ready in under 3 minutes

Why a new approach?

At a regional: know the toolkit, don't blunder the easy half.

At a national championship:

- **Everyone** solves the easy half — rank is decided in the medium-hard band
- Medium-hard = advanced technique **+** a non-obvious observation
- Implementations of 60–150 lines must work first try
- All teams are strong → the **system** is the difference

Calibration (rough)

Signal	Regional finalist	Championship contender
CF rating (individual)	~1700–2000	~2100–2500+
Div. 1 round	A, sometimes B	A–B reliably, C often
Regional set	6–8 solved	9+ solved

Set **process goals** ("upsolve everything ≤ rating+300 within a week"), not outcome goals ("reach 2400").

Four skills that separate teams

1. **Recognition speed** — "this is min-cost flow" within minutes
2. **Implementation reliability** — 120 lines, zero bugs
3. **Proof habit** — 3-line sketch before coding a greedy
4. **Coordination** — the computer is the bottleneck

Every lecture in this track trains 1 and 2. This lecture is about 3 and 4.

The weekly loop

Day	Activity
1	Lecture + implement templates from scratch
2–3	Topic problems (5–8) individually
4	**Team virtual contest** — 5 h, one computer
5	Post-mortem + upsolving
6–7	Individual round (CF/AtCoder) + upsolving

Virtual contests that teach

- Real sets at the right level: past NAC, NWERC, CERC, Asia regionals, WF
- Sources: Kattis problem sources, Codeforces Gym
- Full simulation: one PC, printed notebook, 5 h, frozen last hour
- **One person keeps a timestamped log**:
  read / assigned / submitted / verdict

The log is the raw data for the post-mortem.

Upsolving protocol

1. Think **30–60 min** alone before any editorial
2. Read only the **first hint**, try again
3. Write the solution **from an empty file** — never copy
4. Log: technique + **the observation you missed**
5. Missing technique? → notebook entry

Rule of thumb: upsolve everything solved by ≥ 1/4 of the teams above you.

The upsolve log

Contest	Problem	Technique	Observation missed	Notebook?
NWERC'xx	H	Li Chao	cost is linear in x after fixing j	added
CF 9xx	D	centroid	"count paths of length k"	—

Review monthly. Repeated rows = next training topic.

Post-mortem (45 min, within 24 h)

- **Time lost** — gaps in the log; 90-min problems; idle keyboard
- **Penalty** — each WA: catchable by stress test? overflow? misread?
- **Missed** — which unsolved problem was solvable *today*?
- **One change** — agree on exactly one process change

Five simultaneous changes teach you nothing.

The golden rule of one computer

> The keyboard is only for typing code that is already designed.

- Coder stops typing > 1 minute → hand over the machine
- Debug on **printouts**, not on screen
- Keep an explicit **queue** on paper: "after F: B codes K (~25 min)"

Opening the contest

```
0:00  A: template, compile script, stress.sh   (2–3 min)
      B: reads from the front    C: reads from the back
      → one-line summary + difficulty guess per problem
0:05  A codes the first easy problem
0:20  everyone has seen the set; scoreboard confirms easy ones
```

Adapt to your team — but decide it **before** the contest.

During the contest

- Easy problems: **solo**. Hard problems: **pair** and agree on a proof sketch
- Specialists own areas (geometry, strings) — but everyone codes basics
- WA not obvious in 2 min → print, hand over keyboard, stress test
- Brute force written by a **different** person than the solution
- Last ~45 min: no new problems unless the solution is clear

Stress testing — the kit

```
for f in sol brute gen; do g++ -std=c++17 -O2 -Wall -o "$f" "$f.cpp"; done
for ((i = 1; i <= ${1:-1000}; i++)); do
    ./gen "$i" > in.txt
    ./sol < in.txt > out_sol.txt
    ./brute < in.txt > out_brute.txt
    if ! cmp -s out_sol.txt out_brute.txt; then
        echo "MISMATCH on seed $i"; cat in.txt; exit 1
    fi
done
echo "all tests passed"
```

Ready-made → 3 minutes, not 15.

The generator

```cpp
#include <bits/stdc++.h>
using namespace std;
// usage: ./gen <seed>
int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));
    auto rnd = [&](int lo, int hi) {
        return uniform_int_distribution<int>(lo, hi)(rng); };
    int n = rnd(1, 8);                          // SMALL tests
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(-10, 10) << " \n"[i == n - 1];
}
```

Seed from argv → every failure is reproducible.

Demo: buggy Kadane

```
cur = max(0LL, cur + x); best = max(best, cur);   // best starts at 0
```

```
MISMATCH on seed 7
--- input          --- sol     --- brute
1                  0           -6
-6
```

All-negative arrays: the "empty subarray" bug. Found in < 1 s.

The template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "] = ", dbgPrint(__VA_ARGS__)
// ... variadic dbgPrint (see notes)
#else
#define dbg(...) ((void)0)
#endif
int main() { cin.tie(nullptr)->sync_with_stdio(false); /* ... */ }
```

Local debug: `-DLOCAL -fsanitize=address,undefined -D_GLIBCXX_DEBUG`

The team notebook (TRD)

- WF rule: up to **25 single-sided pages** (check your contest's rules)
- Include: long, error-prone, occasionally needed code
- Exclude: BFS, DSU, binary search — anything you write in 5 lines
- Model: **KACTL** (KTH) — learn from it, but own every line

What goes in (this track builds it)

Area	Entries	Lecture
Data structures	persistent segtree, implicit treap, Li Chao	2–3
Trees	HLD, centroid decomposition	4
Flows	Dinic, MCMF, Hopcroft–Karp, lower bounds	5
Strings	SA + LCP, suffix automaton	6
Math	NTT, Gauss, XOR basis	7
Geometry	hull, half-planes, calipers	8
Graphs	2-SAT, bridges, Euler	9

Notebook rules

1. **Every entry tested**: judge AC + stress test; problem ID in a comment
2. **Uniform interface**: indexing stated on line 1, `ll` for sums
3. **Dense but readable** — no clever macros
4. **Complexity + preconditions** in the header comment
5. **Typed twice** by every member — it's a memory aid, not a crutch

Common mistakes

- Only individual training — no coordination practice
- Editorial after 10 minutes → illusion of learning
- Untested notebook code you *trust*
- Thinking at the keyboard
- Skipping post-mortems → same mistake every week
- Re-reading code for 30 min instead of a 3-min stress test
- Slow on easy problems — penalty decides ties

Practice this week

- Kit: type template + gen + stress from memory; CSES 1643
- Stress-test before submitting: CSES 1085 Array Division
- First notebook page: lazy segtree on CSES 1735
- Team virtual: a past NAC / NWERC set from Kattis or CF Gym
- One CF Div. 1/2 + one AtCoder ARC, upsolved

Summary

- Championships are won in the medium-hard band, by **systems**
- Loop: learn → implement → contest → upsolve → review
- Post-mortem → exactly **one** change per week
- Keyboard only for designed code; debug on paper
- Notebook: tested, uniform, typed by everyone
- Next: **Lecture 2 — Advanced data structures**